

Answr — Technical Documentation

Everything a technical team needs to understand, run, deploy, and extend the Answr platform.

Next.js 16 · React 19 · TypeScript · ~26,600 LOC · 73 routes · 16 API endpoints · 38 library modules · Last updated 21 Aug 2026

Contents

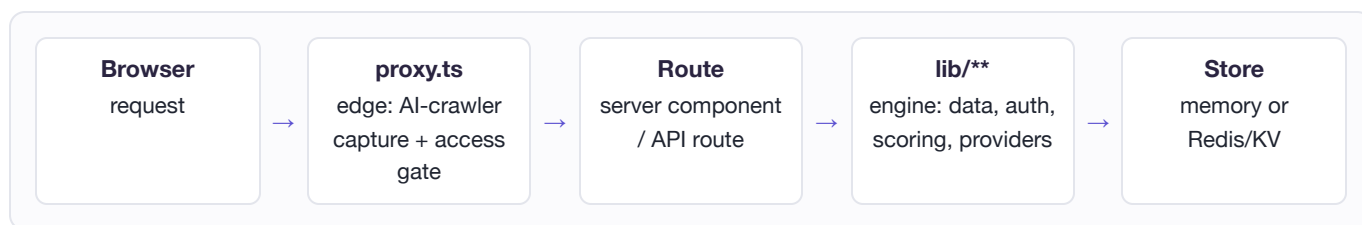
- | | |
|--|-------------------------------|
| 1. System overview & architecture | 9. Running locally |
| 2. Technology stack | 10. Deployment |
| 3. Repository structure | 11. Security model |
| 4. Application surfaces & routing | 12. How to extend it |
| 5. The engine (data & logic layers) | 13. Build, quality & testing |
| 6. API surface | 14. Known limitations & notes |
| 7. Data model & storage | 15. Key-file reference |
| 8. Configuration & environment variables | |

1 · System overview & architecture

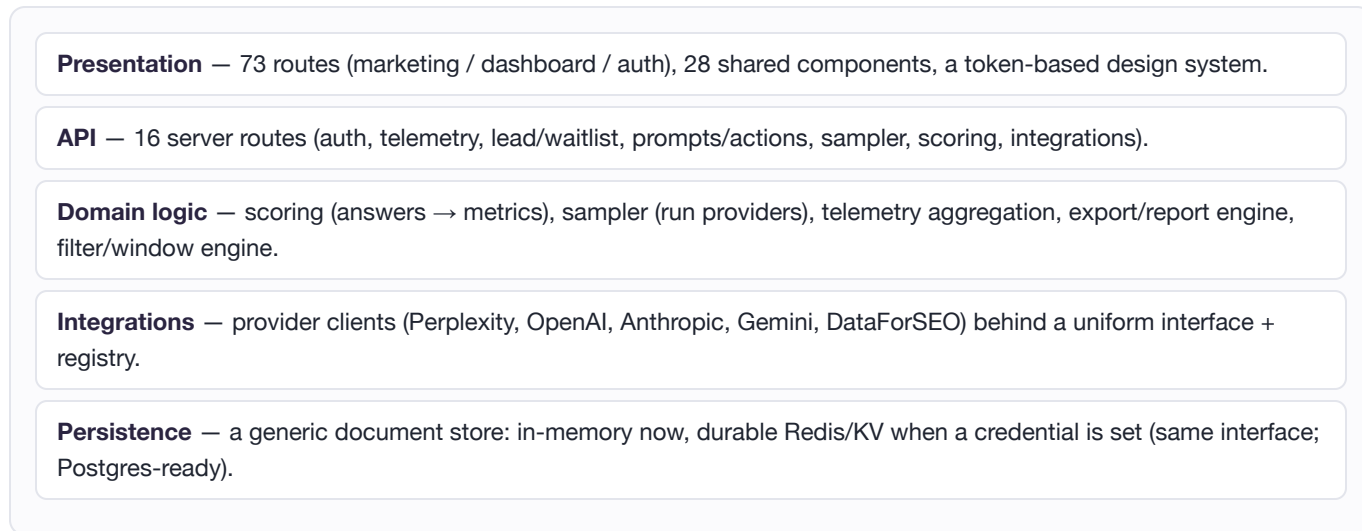
Answr is an **Answer Engine Optimization (AEO)** analytics platform: it measures how a brand appears in AI assistant answers (ChatGPT, Claude, Gemini, Perplexity, Google AI Overviews), traces the citations behind those answers, tracks AI crawlers hitting a site, and turns gaps into a scored action queue.

It is a single **Next.js 16 App Router** application (one deployable) with three concerns: a public marketing site, a gated analytics dashboard, and a backend "engine" of server routes and library modules. Pages are React **server components** by default; interactivity lives in small `"use client"` leaf components. There is no separate backend service — API routes and the engine run inside the same Next.js deployment.

Request flow



Engine layers



Design principle throughout: **works in-memory now, becomes production-durable when a storage key is added** — no code change. Provider- and storage-dependent features are written and wired but inert until a key is present, so the platform runs end-to-end with zero external accounts.

2 · Technology stack

Layer	Technology	Notes
Framework	Next.js 16 (App Router)	Server components, route groups, the new Proxy (middleware) convention
UI	React 19 · TypeScript 5	Type-clean (<code>tsc --noEmit</code> passes)
Styling	Tailwind v4 + CSS variables	Token-based design system in <code>app/globals.css</code> ; inline styles from the design canvases
Charts / maps	d3-geo, topojson-client, world-atlas	Region map; other charts are hand-built SVG/Canvas
Icons	simple-icons	Brand/tech logos with a graceful fallback chain
Runtime deps	8 total	Deliberately lean — no UI kit, no state library, no ORM lock-in
Hosting	Vercel + GitHub auto-deploy	Node 22.x pinned (<code>engines</code> + <code>.nvmrc</code>)
Optional durable store	Upstash / Vercel Redis (REST)	Spoken over plain <code>fetch</code> ; no SDK dependency

3 · Repository structure

```
answr/
├─ app/
│  └─ (marketing)/          31 public pages (home, product/*, industries/*, pricing, legal,
waitlist...)
│  └─ (dash)/app/          36 dashboard screens (overview, insights, citations, agents,
settings...)
│  └─ (auth)/              login, signup, reset, 3-step onboarding
│  └─ api/                 16 server routes
│  └─ globals.css           design-system tokens (colors, type, spacing)
│  └─ layout.tsx            root layout (metadata, skip link, referral snippet)
│  └─ robots.ts / sitemap.ts / opengraph-image.tsx
├─ components/
│  └─ app/                 dashboard shell (Sidebar, Topbar, charts, KpiCard, CommandK...)
│  └─ marketing/           Nav, Footer, LogoWall, LegalDoc
│  └─ ui/                  shared UI (buttons, toaster, sparkline, filter pills...)
├─ lib/
│  └─ db/                  persistence layer (index.ts store + entities.ts)
│  └─ auth/                sscript auth + sessions
│  └─ scoring/             answers → metrics
│  └─ sampler/             nightly run + answer store
│  └─ providers/           Perplexity/OpenAI/Anthropic/Gemini/DataForSEO clients + registry
│  └─ telemetry/           AI-crawler capture store (memory + KV)
│  └─ export/              CSV report engine
│  └─ filters/             date/platform windowing
│  └─ data/                curated demo ("Nike") fixtures
│  └─ metrics.ts           31-metric dictionary (definitions + formulas)
│  └─ gate.ts              demo access gate + cookie names
│  └─ bots.ts              AI crawler / referrer catalog
├─ proxy.ts                edge: AI-crawler capture + access gate
├─ vercel.json             nightly cron → /api/runs/execute
└─ (docs: HANDOFF, INTEGRATIONS, METRICS, PLATFORM_OVERVIEW, ...)
```

4 · Application surfaces & routing

Routing uses Next.js **route groups**, each with its own shared layout:

Group	Shell (layout)	Access
(marketing)	Nav + Footer, 1440px max width	Public
(dash)/app	Sidebar + Topbar, persistent FilterProvider	Gated (see §11)
(auth)	Centered card on a dot-grid	Public (login/signup)

Every request first passes `proxy.ts` (the Next 16 "Proxy" / edge middleware), which (a) matches the user-agent against the AI-crawler catalog and fires a non-blocking capture event, and (b) enforces access to `/app/*`. Pages are server components; the persistent dashboard frame keeps filter and brand selection alive across navigation.

5 · The engine (data & logic layers)

Persistence — `lib/db`

A generic document store with a single `Db` interface (`list / get / put / remove` over named JSON collections). `MemoryDb` runs with zero setup; `KvDb` speaks the Upstash REST protocol via `fetch`. `db()` auto-selects the durable store when `KV_REST_API_*` or `UPSTASH_REDIS_REST_*` is set, else memory — no code change. `entities.ts` defines the typed tables (workspaces, users, sessions, prompts, actions, leads, view events), each tenant-scoped by `workspaceId`. real

Authentication — `lib/auth`

A complete account system on Node `crypto`: `scrypt` password hashing (salted, timing-safe), opaque session tokens, and user/session records in the data layer. Powers `/api/auth/{signup,login,me,logout}`. The dashboard layout validates sessions server-side. real

Scoring — `lib/scoring`

Pure functions that turn sampled answers into the metrics defined in `lib/metrics.ts`: position-weighted visibility score, share of voice, citation counts, owned-citation share, average answer position, and rank-first. Demonstrable at `/api/scoring/preview`. real

Sampler — `lib/sampler`

The nightly job: reads the tracked prompt set, runs every configured provider against each prompt, normalizes answers + citations, and writes runs to the answer store (memory + KV). Triggered by the Vercel cron → `/api/runs/execute`. Refuses to run without a secret so it never spends provider credits unattended. activates with keys

Provider integrations — `lib/providers`

A uniform `AnswerProvider` interface (`sample(prompt) → { text, citations }`) with thin clients for Perplexity, OpenAI, Anthropic, Gemini, and DataForSEO, plus a registry that reports which are configured. Each reads its env key and is inert until one is present. activates with keys

First-party telemetry — `lib/telemetry` + `proxy.ts`

The one pipeline producing real (non-fixture) data today: `proxy.ts` → `/api/ingest` → store → `/app/live`. Captures AI crawlers hitting the site (which never execute JavaScript) and, via `public/snippet.js` + `/api/collect`, AI referrals. real

Export & filters

`lib/export` is a real CSV report engine (titled envelopes, executive summary, per-module specs) used across ~30 controls. `lib/filters` is a deterministic date/platform windowing engine that re-slices charts and honestly discloses the export window. real

6 · API surface

16 server routes under `app/api` . "Gated" = requires the dashboard access cookie.

Route	Method	Purpose	Auth
<code>/api/session</code>	POST / DELETE	Demo gate sign-in / sign-out	Public
<code>/api/auth/signup</code>	POST	Create account (scrypt) + session	Public
<code>/api/auth/login</code>	POST	Authenticate + session	Public
<code>/api/auth/me</code>	GET	Current session user	Session
<code>/api/auth/logout</code>	POST	Destroy session	Session
<code>/api/ingest</code>	POST	AI-crawler telemetry ingest	Secret / origin
<code>/api/collect</code>	POST	AI-referral ingest (snippet)	Secret / origin
<code>/api/telemetry</code>	GET	Telemetry summary + store status	Public
<code>/api/telemetry/test-hit</code>	POST	Fire a synthetic crawler hit (proof)	Public
<code>/api/lead</code>	POST / GET	Capture leads / list them	POST public · GET gated
<code>/api/prompts</code>	POST / GET	Persist / list tracked prompts	Gated
<code>/api/actions</code>	POST / GET	Persist / list action items	Gated
<code>/api/runs/execute</code>	GET / POST	Run the sampler (cron / manual)	Secret
<code>/api/scoring/preview</code>	GET	Scoring engine output (live proof)	Gated
<code>/api/integrations/test</code>	POST	Verify a provider key (one live call)	Gated
<code>/api/waitlist/view</code>	POST / GET	Record view / funnel stats	POST public · GET gated

7 · Data model & storage

All records are JSON documents in named collections, keyed by id, tenant-scoped by `workspaceId` where relevant.

Entity	Collection	Key fields
Workspace	workspaces	id, name, domain
User	users	id, email, passwordHash (bcrypt), workspaceId
Session	sessions	id (token), userId, workspaceId, expiresAt
Tracked prompt	prompts	id, workspaceId, text, intent
Action item	actions	id, workspaceId, title, impact, effort, status
Lead	leads	id, email, company, source, utmSource, referrer
View event	views	id, page, source, ts (waitlist funnel)
Crawler / referral event	telemetry	bot, path, status, verification (first-party capture)
Prompt run / answer	sampler	prompt, per-provider text + citations (sampler output)

Storage backends: in-process memory (default) or Redis/KV (when a credential is set), selected automatically behind the same interface. The design is Postgres-ready — a Postgres implementation of the `Db` interface drops in without touching callers.

8 • Configuration & environment variables

Nothing is required to run the demo. Each variable below turns on a capability.

Variable	Enables
DEMO_PASSWORD	Rotates the demo access passphrase (default <code>answer-demo</code>)
ANSWR_INGEST_SECRET	Authenticates external senders to the ingest endpoints
CRON_SECRET	Arms the nightly sampler (required for it to run)
KV_REST_API_URL / KV_REST_API_TOKEN	Durable store (Vercel Redis)
UPSTASH_REDIS_REST_URL / _TOKEN	Durable store (Upstash — alternative)
PERPLEXITY_API_KEY	Perplexity Sonar sampling lane
GEMINI_API_KEY	Gemini grounded sampling lane
DATAFORSEO_LOGIN / DATAFORSEO_PASSWORD	Google AI Overviews + demand data
ANTHROPIC_API_KEY	Claude lane + sentiment classification
OPENAI_API_KEY	ChatGPT lane
RESEND_API_KEY / LEAD_NOTIFY_EMAIL / LEAD_FROM_EMAIL	Lead + transactional email
*_MODEL, DATAFORSEO_LOCATION / _LANGUAGE	Optional per-provider overrides (sensible defaults)

Verified: a `grep` for `process.env` shows these are the only variables the code reads. No secret is committed to the repository.

9 • Running locally

Prerequisites: Node 22.x. No database or keys needed for the demo.

```
npm install
npm run dev      # dev server (defaults to port 3000; project launch config uses 3201)
npm run build    # production build
npx tsc --noEmit # type-check
```

Open the app, go to `/login` , and sign in with the demo passphrase (default `dana@nike.com` / `answer-demo`) to reach the dashboard.

10 · Deployment

Deployed on **Vercel** with **GitHub auto-deploy**: a push to `main` builds and promotes to production; branches get preview URLs. `vercel.json` schedules the nightly sampler cron. Node is pinned to 22.x. To deploy under a new account: connect the repo to a Vercel project, set the desired environment variables, point the domain, and push.

11 · Security model

- **Access gate** — `/app/*` is protected. `proxy.ts` does an edge-safe cookie presence check; the dashboard layout then validates for real, granting access by *either* the demo passphrase *or* a valid account session, and bouncing invalid sessions.
- **Authentication** — script password hashing (salted, timing-safe), opaque session tokens, HTTP-only cookies. A security review is recommended before production reliance.
- **Ingest hardening** — the telemetry ingest endpoints accept a shared secret (`ANSWR_INGEST_SECRET`) and check origin; set the secret in production.
- **Sampler safety** — the sampler refuses to run without `CRON_SECRET` , so provider credits are never spent unattended.
- **Secrets** — none are committed; all live in the deployment environment.

12 · How to extend it

Task	How
Add an AI provider	Implement the <code>AnswerProvider</code> interface in <code>lib/providers</code> , add it to the registry array — the Integrations UI and sampler pick it up automatically.
Add / change a metric	Edit <code>lib/metrics.ts</code> (definition + formula) and the corresponding function in <code>lib/scoring</code> .
Swap to Postgres	Implement the <code>Db</code> interface (<code>lib/db</code>) over Postgres; no caller changes.
Add a page	Create a <code>page.tsx</code> in the appropriate route group; it inherits that group's shell.
Wire scoring into a dashboard	Read accumulated runs from the answer store, call <code>scoreRuns()</code> , and render in place of the <code>lib/data/*</code> fixture for that screen.
Turn on real account access	Make <code>lib/auth</code> sessions the sole gate for <code>/app</code> (JWT-at-edge or the existing node layout guard) and retire the demo passphrase.

13 · Build, quality & testing

- **Type-safe** — `tsc --noEmit` passes with zero errors; no page is a client component.
- **Clean build** — `next build` generates 78 static pages with no warnings.
- **Honesty layer** — ~110 inert controls each state what they would do on a live workspace, rather than failing silently — an unusually disciplined pattern that aids both users and maintainers.
- **Accessibility** — landmarks, a skip link, keyboard focus rings, ARIA on interactive widgets, reduced-motion support.
- **SEO** — native `robots.txt`, `sitemap.xml`, Open Graph image, and per-page metadata.

There is no automated test suite; verification to date has been type-checking, clean builds, and manual/functional checks. Adding tests around the engine (scoring, providers, db) is a reasonable early investment for the new owner.

14 · Known limitations & notes for a new team

- **Demo vs live data** — most dashboard numbers are the curated "Nike" fixture; they become real once providers are connected and scoring is wired into the views (see `LAUNCH_ROADMAP.md`).
- **In-memory default** — without a storage key, data resets on serverless cold starts and is not shared across instances. Set a Redis/KV key for durability.
- **Provider clients** — written against documented endpoints but not run against live keys; verify each with a real key (the Integrations "Test connection" button exists for this).
- **Access gate** — the demo passphrase still governs `/app`; switching to real account sessions is the documented last wiring step.
- **Dev-only quirk** — in local dev, server-rendered pages and API routes are separate module graphs, so in-memory writes via an API route aren't visible to a page render; this disappears with a durable store and in production.
- **Conversation / demand panel** — panel-grade conversation metrics require a licensed data contract; until then they should be labeled "modeled / beta."

15 · Key-file reference

File	Role
proxy.ts	Edge: AI-crawler capture + access gate
app/(dash)/app/layout.tsx	Dashboard shell + real session validation
lib/db/index.ts · entities.ts	Persistence store + typed tables
lib/auth/index.ts	script auth + sessions
lib/scoring/index.ts	Answers → metrics
lib/sampler/run.ts · store.ts	Nightly run + answer store
lib/providers/*	Provider clients + registry
lib/telemetry/*	First-party crawler/referral capture
lib/metrics.ts	31-metric dictionary (definitions + formulas)
lib/export/report.ts	CSV report engine
HANDOFF.md	Step-by-step activation guide
INTEGRATIONS.md · METRICS.md	Integration plan · metric reference

This document accompanies the Answr platform for acquisition diligence. For the full feature inventory see PLATFORM_OVERVIEW.md; for activation steps see HANDOFF.md; for the assets transferring with the sale see the Assets & IP schedule.